

Assignment 2- Case Study (8995-G level Task)

Stage 5 Tutorial Activities

Architecture and Responsibility

Week 8 | 60 minutes

Activity 1 - Where Does This Belong?

Responsibility	Layer	Reason
Read menu input	Presentation	It directly asks the user for information.
Check appointment status transition	Domain	This is a core business rule for the clinic.
Coordinate booking use case	Service	It controls the step-by-step process of booking.
Execute SQLite INSERT	Persistence	It talks to the database to save things.
Format confirmation message	Presentation	It makes the text look nice for the user to read.
Find appointment by ID	Repository	It fetches data without the main code needing to know SQL.

Activity 2 - Architecture Smell Hunt

A SmartCare file contains input(), SQL, appointment conflict rules, printing and validation. Identify at least five architecture problems and propose a layer for each responsibility.

1. Using the input() function mixes user interface tasks with the main code, so it belongs in the Presentation Layer.
2. Writing SQL database commands directly in the main file is a bad design, so they must be moved to the Persistence Layer.
3. Rules about appointment conflicts are core business logic, so they should be placed in the Domain Layer.
4. Printing text to the screen is a user interface job, so it needs to be handled by the Presentation Layer.
5. Checking if data is valid is a business rule, so this responsibility should be moved to the Domain Layer.
- 6.

Activity 3 - SOLID Without Overengineering

ClinicManager handles every use case. Which principle is threatened?

The principle that is threatened is the Single Responsibility Principle (SRP) because the class is trying to do too many jobs at once.

AppointmentService imports sqlite3 directly. What dependency concern exists?

The dependency concern that exists is the Dependency Inversion principle (DIP) because our main logic relies directly on a specific database tool.

A repository interface has 20 methods, but a client needs two. What concern exists?

The concern that exists is the Interface Segregation Principle (ISP) because we shouldn't have to load 18 methods we don't even need to use.

Should every class have an interface? Explain.

No, not every class should have an interface because that is overcomplicating things. We only need them when we want to hide messy details like a database.

Activity 4 - AI Architecture Critique

AI proposes microservices, an event bus, six interfaces and a dependency-injection framework. Decide what to reject, defer or keep using current requirements.

Reject: I would reject the microservices and event bus. The client specifically asked for a small, simple system, and this is way too complicated.

Defer: Rather than having multiple interfaces, we should wait until we need them since as of now, we only need one interface.

Keep: Using a simple repository interface to hide the database code.